

ASIC Lab Handbook（日本語版）

4-cell, 6-bit Wilkinson waveform-digitizer prototype

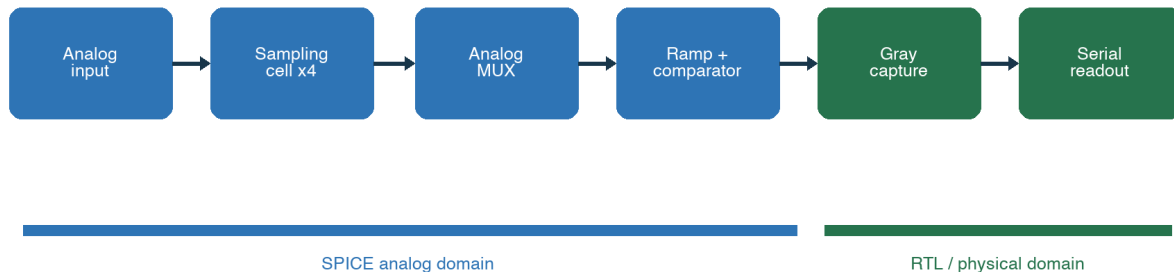


図1 本プロトタイプの信号経路。青はアナログ、緑はデジタル領域。

Version 1.1 | 2026-08-13

Repository: github.com/sykeisuke/asic_rd

このハンドブックの目的

この教材のゴールは、最高性能の ASIC を一度で作ることではありません。仕様、回路図、SPICE、RTL、協調検証、配置配線、サインオフ、GDSII という一連の流れを、誰でも再現できる形で通すことです。利用者は GitHub の設計ファイルを読み、同じコマンドで結果を再生成し、次の設計判断を説明できる状態を目指します。

本書がこの project の唯一の正式教材です。前半で環境と回路の読み方を学び、Lab 0 から順に手を動かします。各 Lab では、目的、入力、出力、観測点、合格条件を確認してから次へ進みます。

最重要原則結果のスクリーンショットだけを信用しない。入力ファイル、実行コマンド、生成物、合否基準の 4 点を常に対応づけます。

学習成果

- GF180 PDK を使った MOS 特性と基本アナログ回路を説明できる。
- サンプル・ホールド、ランプ、コンパレータから Wilkinson ADC の変換原理を説明できる。
- SPICE イベントを RTL へ渡す境界と、Gray code を用いた CDC 対策を説明できる。
- RTL-to-GDS フローの各成果物と DRC/LVS/STA の意味を説明できる。
- prototype と tape-out-ready design の違いを、未解決項目を含めて説明できる。

推奨の進め方

段階	Lab	成果
基礎	0 – 2	環境、PDK、MOS、sampling
変換器	3 – 5	comparator、ramp、1/4-cell Wilkinson
デジタル	6 – 8	controller、co-sim、CDC、serial
物理実装	9	netlist から GDSII
設計レビュー	10	証拠、blocker、次版仕様

0. 初学者のための読み方

ASIC 設計では、回路、software、半導体 process、測定という言葉が同時に登場します。最初から全用語を暗記する必要はありません。まず『何を入力し、どの道具で処理し、何が出力されるか』の 3 点だけを追い、波形や log を見ながら意味を増やします。

最初に区別する 4 つ

言葉	簡単な意味	この project の例
回路図 / schematic	部品と接続を人が読む図	nmos_dc.sch, sampling_cell.sch
netlist	simulator が読む部品と接続の文章	*.spice, Xschem 生成 netlist
RTL	clock ごとの digital 動作を記述する文章	*.v, synthesizable Verilog
layout	silicon 上の layer 形状	DEF, GDSII

model と実物を混同しない

- SPICE model は transistor の電氣的近似。nominal simulation が通っても silicon 保証ではない。
- RTL simulation は logic と clock 順序を確認する。transistor-level analog 挙動は含まない。
- GDS は形状データ。package、bond wire、PCB、測定器まで含めて初めて実験 system になる。

数値と単位

接頭語	倍率	例
m	10^{-3}	1 mV = 0.001 V
μ / u	10^{-6}	1 μ s = 0.000001 s
n	10^{-9}	1 ns
p	10^{-12}	1 pF, 1 ps

実験ノート 各 Lab で、commit hash、command、開始時刻、終了時刻、PASS/FAIL、主要 artifact、気づきを記録します。再現できない成功は設計結果として弱いからです。

1. 使用ツールを理解する

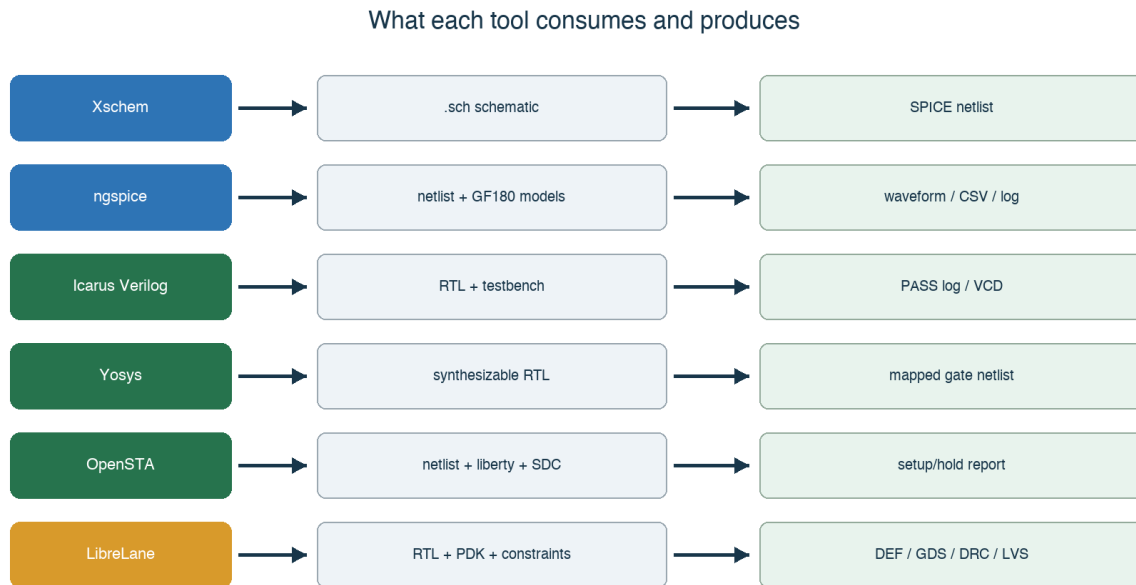


図 2 各 tool の入力と出力。Makefile はこの連鎖を短い command で実行する。

Tool	役割	利用者が直接行うこと
Docker Desktop	固定 Linux 環境を Mac 上で動かす	起動、container 状態と disk 容量確認
Make	複数 command を target 名へまとめる	make nmos-dc 等を実行
Xschem	analog 回路図編集と SPICE netlist 生成	symbol 配置、wire、property 編集、netlist 確認
ngspice	transistor-level simulation	log、測定値、CSV 波形を読む
AWK	log/CSV の抽出と自動合否判定	script が何を PASS 条件にするか読む
Icarus Verilog	RTL simulation	iverilog で compile、vvp で実行、VCD 確認
Yosys	RTL synthesis	mapped netlist と cell 統計を読む
OpenSTA	static timing analysis	SDC constraint と setup/hold report 確認
LibreLane	RTL-to-GDS orchestration	config.yaml、metrics、final GDS 確認
KLayout/Magic/Netgen	layout 閲覧、DRC/LVS。analog layout で本格使用	layer、pin、DRC marker、LVS mismatch 確認

現在と将来 KLayout/Magic/Netgen は analog layout 段階で直接操作します。現時点の digital physical flow では LibreLane が複数 backend tool を統括しています。

2. Makefile の裏側

make は simulator ではありません。Makefile で target 名を shell script へ対応づける入口です。たとえば make nmos-dc は scripts/run-nmos-dc.sh を呼び、その script が Docker 内で Xschem と ngspice を実行します。

```
make nmos-dc -> scripts/run-nmos-dc.sh -> docker run -v <repo>:/foss/designs -> xschem -n ... nmos_dc.sch -> ngspice -b ...spice -> work/nmos_id_vds.csv
```

script を読む順番

1. `set -eu` を確認する。`error` や未定義変数で停止する設定。
2. `eda-common.sh` から `EDA_IMAGE`、`PROJECT_ROOT`、`Docker CLI` を読む。
3. `docker run` の `-v` を見て、Mac 側 repo と container 内/`foss/designs` の対応を理解する。
4. container 内の `cd` 先を確認する。
5. 実行 tool と option を読む。`-b` は batch、`-o` は log 出力など。
6. 最後の `test/grep` を読み、何を `PASS` としているか確認する。

代表的な実行経路

Target	内部 tool	主な artifact
<code>make nmos-dc</code>	Xschem → ngspice	netlist, raw, CSV
<code>make four-cell-cosim</code>	ngspice → grep/AWK → iverilog/vvp	analog_stimulus.vh, cosimulation.log
<code>make digital-top</code>	iverilog/vvp → Yosys → OpenSTA	VCD, mapped.v, timing log
<code>make digital-physical</code>	LibreLane/OpenROAD ecosystem	GDS, DEF, metrics, reports

自走のコツ `make` が失敗したら `target` 名だけを眺めず、対応する `scripts/run-*.sh` を開き、最後に成功した `command` と最初に失敗した `command` を特定します。

3. アナログ設計の基礎

Analog design: from device bias to sampled voltage

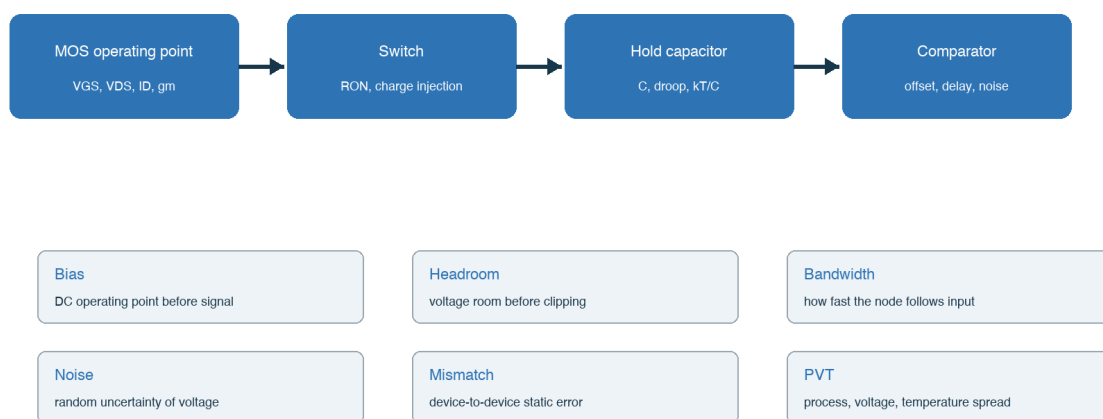


図3 device の bias から sampling 誤差までを一つの因果関係として考える。

digital 信号は 0/1 へ抽象化できますが、analog node は連続した電圧・電流・時間を持ちます。回路を読むときは、まず DC operating point、次に小さな signal への応答、最後に large transient と noise/mismatch の順で考えると整理しやすくなります。

3.1 MOS transistor の最低限

量	意味	確認方法
VGS	gate-source 間電圧。channel 形成を支配	DC sweep
VDS	drain-source 間電圧。動作領域と headroom に関係	I-V curve
ID	drain 電流。bias/power/速度を決める	operating point
gm	gate 電圧変化を電流へ変える強さ	op result または微分
ro	saturation 領域の有限出力抵抗	Id-Vds の傾き
W/L	device geometry。電流、容量、mismatch を変える	schematic property

body 端子 MOS は 4 端子 device です。bulk/body 接続を省略して考えると、body effect や parasitic diode を見落としてします。

3.2 Bias、headroom、gain、bandwidth

Bias は signal がないときの静止状態です。transistor が意図した領域に入り、全 node が電源範囲内にあることを先に確認します。Headroom は signal が振れても device が必要な電圧を保てる余裕です。

- DC operating point: node 電圧と branch 電流が妥当か。
- Gain: 小さな入力変化が出力へ何倍伝わるか。
- Bandwidth: gain が低下せず追従できる周波数範囲。
- Slew rate: large signal で出力が変化できる最大速度。
- Settling: 目標値の許容誤差内へ入るまでの時間。

RC と sampling

switch の ON 抵抗 R_{ON} と hold capacitance C は時定数 $\tau = R_{ON} \cdot C$ を作ります。一次近似では誤差は $\exp(-t/\tau)$ で減少します。sampling pulse が短すぎると、hold node は入力へ十分 settle しません。一方 C を大きくすると kT/C noise と droop は改善しやすいものの、acquisition は遅く、面積も増えます。

C を大きくすると	良くなりやすい	悪くなりやすい
効果	kT/C noise、droop、charge sharing 感度	acquisition time、switch 負荷、面積

簡単な計算例

$R_{ON} = 2 \text{ k}\Omega$ 、 $C = 1 \text{ pF}$ なら $\tau = 2 \text{ ns}$ です。0.1% 程度まで settle させるには約 7τ 、すなわち約 14 ns が一つの目安です。これは first-order 近似であり、実回路では signal-dependent R_{ON} や寄生容量を SPICE で確認します。

3.3 誤差、noise、PVT、Monte Carlo

分類	例	検証
Systematic error	comparator offset、ramp slope error	corner/parameter sweep、calibration
Random noise	thermal noise、kT/C	noise/transient noise、統計
Mismatch	同じ寸法の MOS 間の差	Monte Carlo mismatch
PVT	process、supply、temperature	corner matrix
Parasitic	layout R/C、coupling	PEX 後 simulation

Nominal simulation は typical model、代表電源、代表温度での一例です。Tape-out 判断には、fast/slow device corner、電源 min/max、温度範囲、mismatch seed、extracted parasitic を組み合わせた verification plan が必要です。

初学者が波形を見る順序

1. 電源と ground が期待値か。
2. 入力刺激の振幅・位相・立上りが想定どおりか。
3. 内部 node が rail へ張り付いていないか。
4. 出力の極性が合っているか。
5. 時間軸と電圧軸の単位が合っているか。
6. 測定 cursor だけでなく前後の transient も見る。

重要波形の形を先に眺め、後から都合のよい測定点を探さない。先に測定定義と合否条件を書きます。

4. モジュールの接続

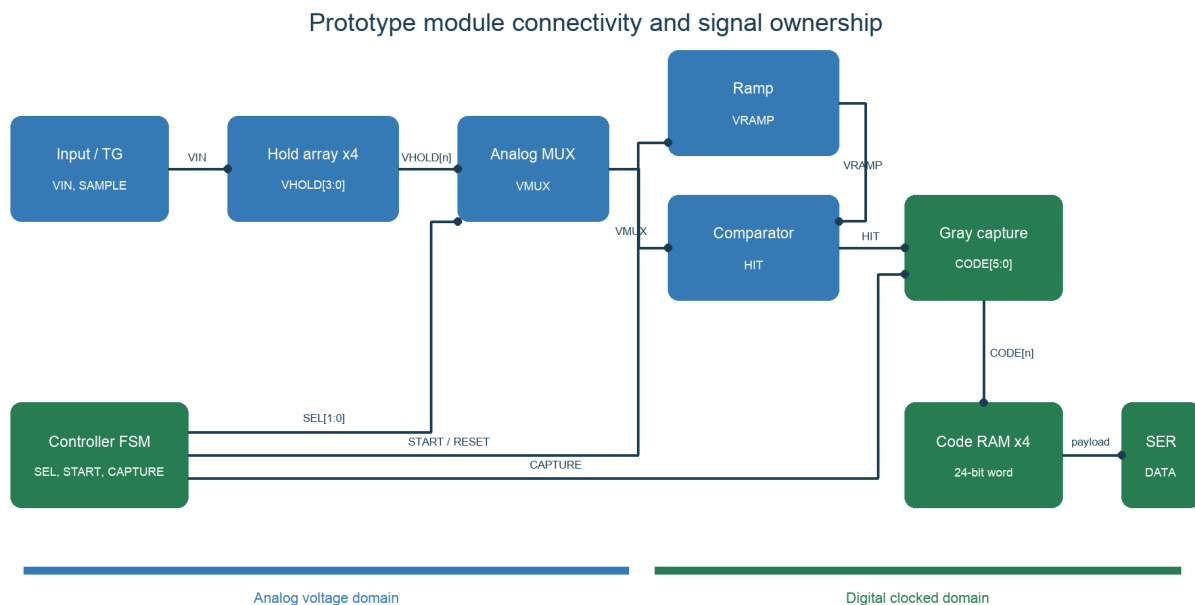


図4 信号名付き module 接続。controller が時間順序を作り、analog crossing を Gray capture が code へ変える。

Signal	Producer	Consumer	種類・意味
VIN	signal source/pad	sampling transmission gate	analog voltage
SAMPLE[3:0]	controller/clock generation	sampling cells	digital control
VHOLD[3:0]	hold capacitors	analog MUX	stored analog voltage
SEL[1:0]	controller	analog MUX	selected cell index
VMUX	analog MUX	comparator	selected held voltage
VRAMP	ramp generator	comparator	time reference voltage
HIT	comparator	Gray capture/controller	asynchronous crossing event
CODE[5:0]	Gray capture	code registers	one cell result
DATA	serializer	FPGA	24-bit payload, serial

4.1 1 回の変換を時系列で追う

1. Track: SAMPLE[n]=1. switch が ON になり VHOLD[n] が VIN へ追従する。
2. Hold: SAMPLE[n]=0. switch を OFF にし、capacitor へ電圧を保存する。
3. Select: SEL=n. VHOLD[n] だけを VMUX へ接続する。
4. Settle: MUX 切替による transient が収まるまで待つ。
5. Reset: VRAMP と counter を既知値へ戻す。
6. Convert: ramp を開始し、Gray counter を進める。
7. Compare: VRAMP が VMUX を横切ると HIT edge が発生する。
8. Capture: HIT 近傍の Gray code を保存し binary code へ扱う。
9. Repeat: n=0..3 について繰り返す。
10. Readout: 4×6 bit を 24-bit word へ連結し serial 送信する。

analog/digital 境界で決めること

境界	曖昧だと起きる問題	固定事項
Comparator → capture	±1 以上の誤 code、metastability	edge polarity、synchronization、capture convention
Controller → analog switch	short、charge sharing、settling 不足	non-overlap、pulse width、SEL timing
Code RAM → serializer	cell 順・bit 順が逆	packing order、MSB/LSB first、frame valid

設計レビュー 各 arrow について producer、consumer、voltage domain、clock domain、active polarity、reset state を説明できれば、接続の理解がかなり進んでいます。

5. 全体設計フロー

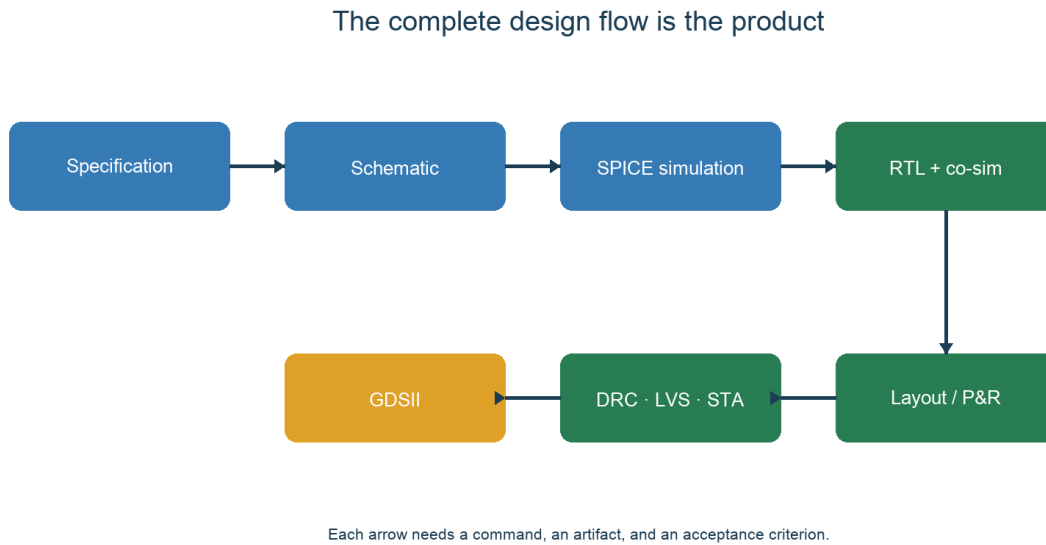


図 5 仕様から GDSII まで。各矢印が検証可能であることが complete flow の条件。

ブロックの役割

ブロック	入力 → 出力	この prototype で確認すること
Sampling cell	VIN + SAMPLE → VHOLD	4 つの異なる電圧を保持できる
Analog MUX	VHOLD[3:0] + SEL → VMUX	選択セルだけを共有 ADC へ接続する
Ramp	START → VRAMP	単調な電圧-時間変換を作る
Comparator	VMUX, VRAMP → HIT	交差時刻をデジタルイベントへ変換する
Gray counter/capture	CLK, HIT → CODE[5:0]	境界付近でも多 bit 同時遷移を避ける
Serializer	CODE x4 → DATA	少ない pad 数で FPGA へ読み出す

今回の仕様 4 storage cells、6-bit 変換、共有 ramp/comparator、24-bit serial readout。速度・分解能・帯域よりも flow の完結性を優先します。

Lab 0 リポジトリを読む

到達目標設計ソースと生成物を区別し、再現コマンドを自分で見つける。

```
git clone https://github.com/sykeisuke/asic_rd.git cd asic_rd git status make check
```

最初に読む 5 ファイル

ファイル	読む理由
README.md	入口、クイックスタート、全体構成

ファイル	読む理由
docs/PROTOTYPE_SPECIFICATION.md	何を作るか、数値仕様、success criteria
docs/TOP_LEVEL_INTERFACE.md	アナログ/デジタル境界と I/O
docs/VERIFICATION_MATRIX.md	要求と test の対応
docs/TAPEOUT_BLOCKERS.md	prototype 後に残る実装リスク

ディレクトリの見方

- simulations/: Xschem/ngspice 用のアナログ testbench と結果。
- digital/: synthesizable RTL、testbench、physical design 設定。
- docs/: 仕様、判断記録、検証表。
- work/ や runs/: 再生成できる中間成果物。設計の source of truth と混同しない。

確認問題

- 現在の HEAD commit を記録し、実験ノートに貼る。
- VERIFICATION_MATRIX の 1 行を選び、要求、test、artifact、判定を説明する。

6. Mac + Docker 開発環境

Mac はホストとして十分に使えます。Linux 向け EDA ツールと GF180 PDK は Docker コンテナ内へ固定し、Xschem など GUI はブラウザ VNC から操作します。これにより環境ごとの macOS 差を小さくします。

1. Docker Desktop を起動し、メイン画面で Engine が Running であることを確認する。
2. リポジトリで make vnc を実行する。
3. ブラウザで <http://localhost:8080/> を開き、ローカル環境で設定した VNC password を入力する。
4. File Manager から /foss/designs/simulations/gf180_nmos_dc/nmos_dc.sch を開く。

```
cd /Users/yeisuke/Desktop/mywork/asic_rd make vnc # Browser: http://localhost:8080/
```

アカウントは必要か

通常のローカル image 実行に Docker Hub アカウントは必須ではありません。ただし private image、rate limit 回避、組織管理を使う場合はログインが必要です。授業では使用 image の digest または tag を固定します。

よくあるトラブル

症状	確認
localhost:8080 が開かない	docker ps、make vnc のログ、port 競合
Xschem に PDK symbol がない	PDK_ROOT とコンテナ内 mount
ファイル変更が見えない	host path と /foss/designs の対応

症状	確認
com.docker.vmnetsd 警告	Docker 公式最新版、残存 helper、macOS Gatekeeper を確認

安全上の注意 Gatekeeper を恒久的に無効化しない。Docker 公式配布物を使い、異常が続く場合は再インストールと helper cleanup を優先します。

6.1 Docker ・ VNC ・ /foss の関係

VNC で接続している先は Docker Desktop の管理画面ではなく、Docker Desktop 上で動く IIC-OSIC-TOOLS Linux container の仮想 desktop です。Browser は localhost:8080 へ接続し、container 内の web/VNC server が Linux desktop 画面を転送します。

Mac -> Docker Desktop -> container: asic-rd-gf180-vnc -> Linux desktop + EDA tools + PDK -> web/VNC server : port 80 -> Browser localhost:8080

質問	答え
IIC-OSIC-TOOLS には何が入る？	Linux user environment、Xschem、ngspice、KLayout、Magic、Netgen、Icarus Verilog、Yosys、OpenSTA、LibreLane/OpenROAD 系、PDK、VNC/web server 等。
一つの巨大 application？	いいえ。独立 tool と PDK を version 固定した Linux image。script や LibreLane が必要な tool を順番に呼ぶ。
新しい環境の/foss は空？	/foss/pdks 等は image が提供。/foss/designs は利用者が clone した repository を Mac から mount する。
編集結果は消える？	/foss/designs 内は Mac 側 repository へ直接書かれるので残る。container 内部だけの file は container 削除で失う可能性がある。
Connected to <id> とは？	起動中 container の desktop session へ browser VNC が接続済みという意味。
同じ環境を再現するには？	同じ Git commit、同じ image tag/digest、同じ Make target を使う。

volume mount

eda-vnc.sh の -v “\$PROJECT_ROOT:/foss/designs:rw” が対応を作ります。PROJECT_ROOT は make vnc を実行した asic_rd repository です。

Host Mac: /Users/.../asic_rd | Docker bind mount Container: /foss/designs

開始手順 git clone 後、必ず asic_rd directory へ cd して make vnc を実行します。/foss/designs が空なら、clone ではなく mount と起動 container を確認します。

7. Xschem を初めて使う

Xschem は回路図 editor であり、simulation engine そのものではありません。symbol と wire から SPICE netlist を生成し、その netlist を ngspice へ渡します。本 project では GUI 確認に加え、script から headless netlisting も行うため、手動操作と自動実行の結果を比較できます。

回路図を開く

1. make vnc を実行し、browser desktop を開く。
2. File Manager で /foss/designs/simulations/gf180_nmos_dc/ へ移動する。
3. nmos_dc.sch を double-click する。関連付けで開かない場合は Xschem を起動して File > Open を使う。
4. 画面上の NMOS symbol、voltage source、ground、simulation command を探す。
5. symbol を選択して property を開き、model 名、W、L、multiplicity を読む。

配線を読むときの規則

- 線が交差しても junction dot がなければ接続でない場合がある。
- node label が同じなら離れた場所でも同一 net として接続される。
- ground/reference node がない SPICE 回路は解けない。
- MOS の D/G/S/B 端子を symbol 向きだけで判断せず property/netlist でも確認する。

変更前に守ること

- 元の testbench を直接壊さず、学習用 copy または Git branch を作る。
- 一度に W、L、bias、stimulus を同時変更しない。
- 変更前の予測を数値または増減方向で書く。
- 保存後に git diff で意図した変更だけか確認する。

ショートカット Xschem の shortcut は version や keymap で差があり得ます。授業ではまず menu 名で操作を共有し、各自の画面下 status/help で shortcut を確認します。

7.1 nmos_dc.sch を開いた後

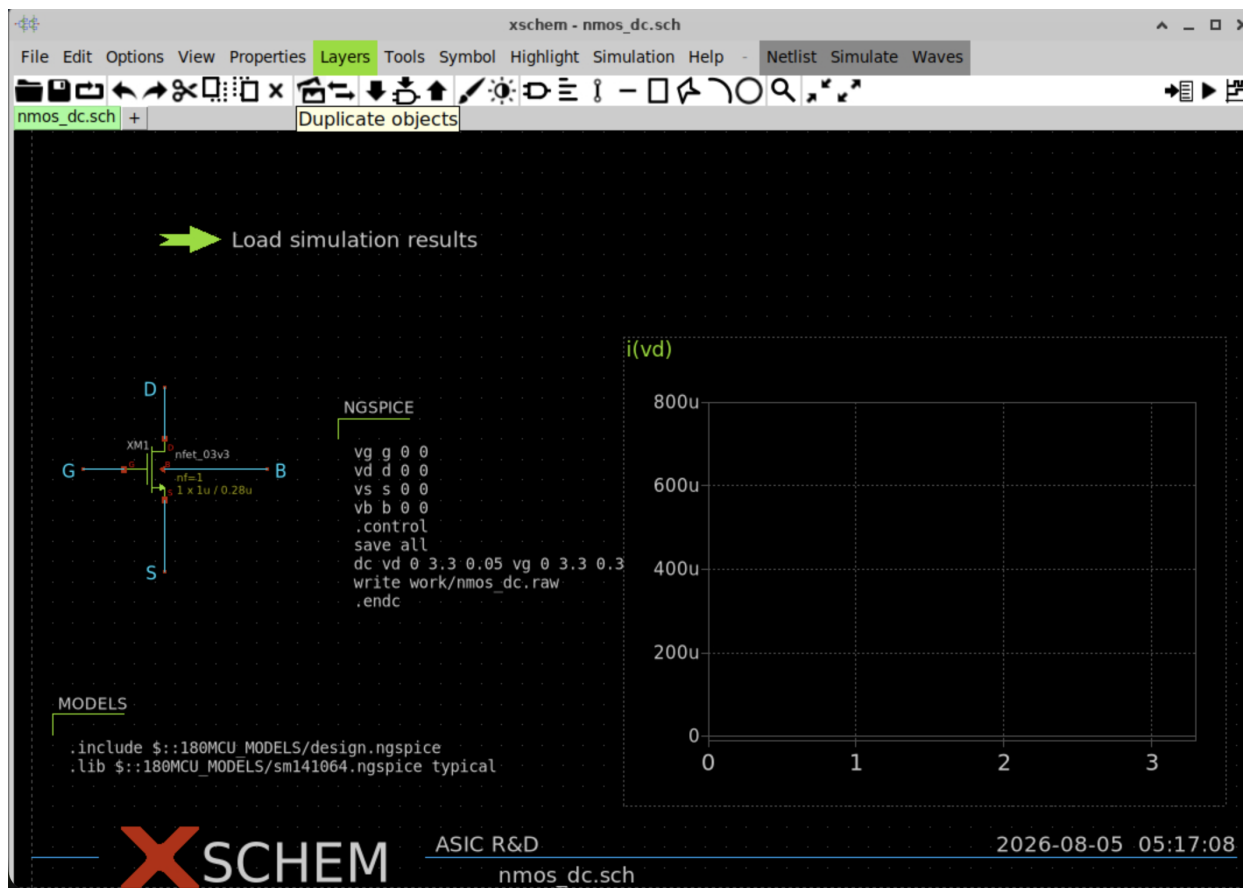


図 6 nmos_dc.sch を開いた直後。右の graph が空なのは RAW 結果をまだ読み込んでいないため。

画面のどこを見るか

場所	内容
左中央	GF180 3.3 V NMOS。G/D/S/B の 4 端子、W=1 μ m、L=0.28 μ m
中央	ngspice command。VDS を 0 – 3.3 V、VGS を 0 – 3.3 V で sweep
左下	GF180 model include と typical library 選択
右	i(vd) -1 *を表示する graph。縦軸は drain current、横軸は VDS
左上の緑矢印	\$netlist_dir/nmos_dc.raw を読み込む launcher

7.2 最初の波形を表示する

1. Mac terminal で repository 直下から make nmos-dc を実行する。すでに実行済みなら work/nmos_dc.raw が存在する。
2. Xschem 画面左上の緑矢印『Load simulation results』を Ctrl + 左クリックする。通常のクリックやダブルクリックは launcher の選択または property 編集になる。

GUI 内で再計算する場合は、回路図を保存してから上部の Netlist、続いて Simulate を押し、最後に launcher を Ctrl + 左クリックする。Xschem は \$netlist_dir で ngspice を起動するため、回路図内の write には work/ を付け

ず write nmos_dc.raw とする。

3. 右側 graph に複数の Id-Vds curve が現れることを確認する。
4. curve が見えなければ Xschem を開いた directory が /foss/designs/simulations/gf180_nmos_dc か確認する。
5. Terminal で ls -lh work/nmos_dc.raw を実行し、file が 0 byte でないか確認する。
6. 必要なら make nmos-dc を再実行し、Xschem を閉じて開き直してから、再度 Ctrl + 左クリックする。

Mac terminal cd /Users/ykeisuke/Desktop/mywork/asic_rd make nmos-dc # Container terminal で確認する場合 cd /foss/designs/simulations/gf180_nmos_dc ls -lh work/nmos_dc.raw

RAW を開けない場合 raw_read(): failed to open ... と表示されたら、Xschem を /foss/designs/simulations/gf180_nmos_dc から起動しているか確認します。古い回路図を開いたまま project を更新した場合は、一度閉じて開き直します。

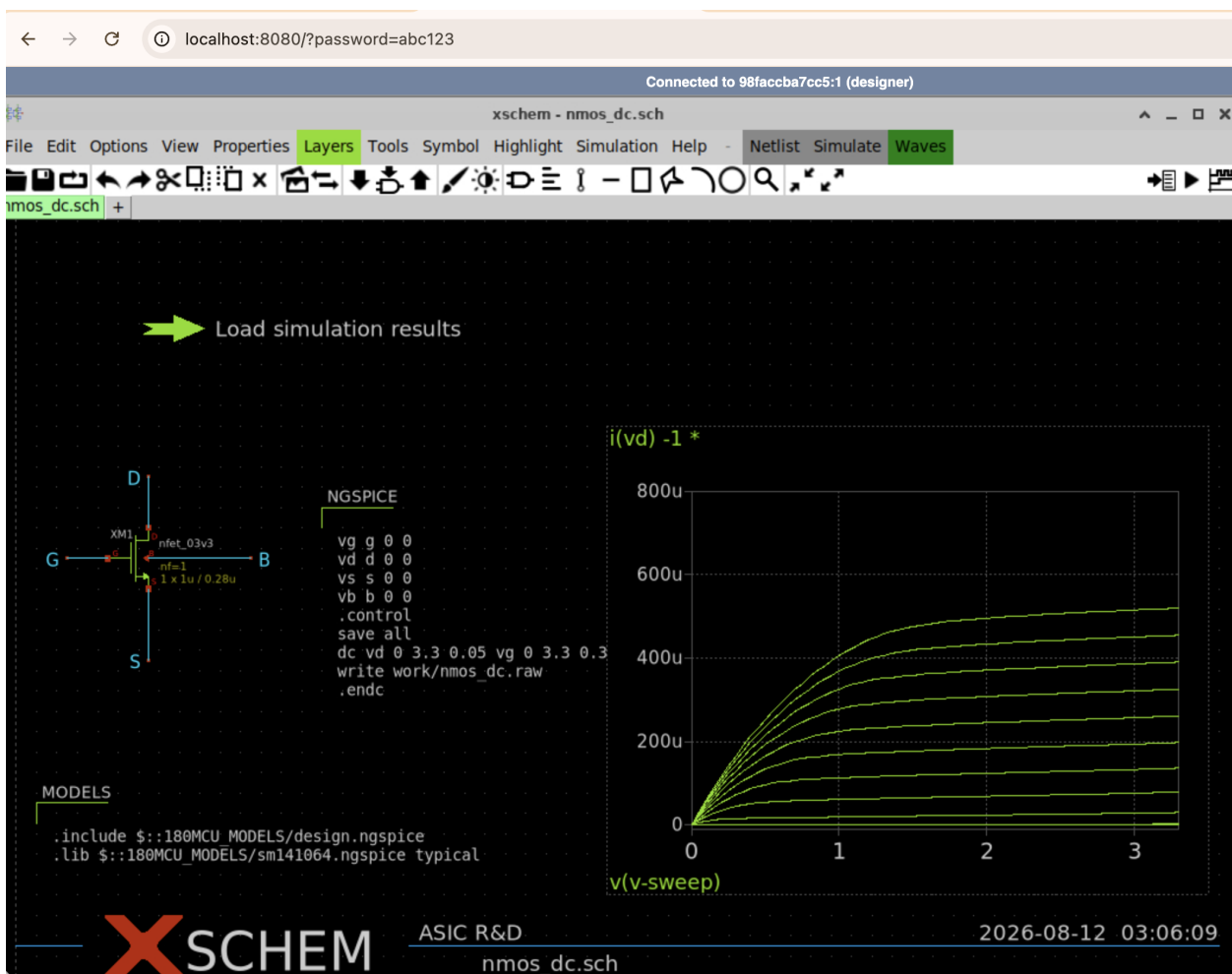


図7 期待する Id - Vds 表示。VGS ごとに複数の curve が現れ、最大電流は約 520 μA。

表示後に読み取ること

- VGS が高い curve ほど drain current が大きい。
- 低 VDS では電流が VDS に強く依存し、VDS 増加後は傾きが小さくなる。
- 縦軸の u は μA。800u は 800 μA。

- SPICE の電圧源電流は source へ流れ込む向きが正なので、NMOS drain current を上向きに表示するには $i(vd) \cdot -1$ とする。Xschem の graph 演算は後置記法 (RPN)。
- graph の curve 数は VGS sweep 点に対応する。
- この結果は typical model の nominal DC sweep であり、PVT 保証ではない。

ここでの到達点自分で RAW を読み込み、curve を表示し、VGS/VDS/ID/W/L/model corner を説明できれば Lab 1 へ進めます。

7.3 Netlist と ngspice を追跡する

make nmos-dc を実行すると、Xschem は nmos_dc.sch から work/nmos_dc_xschem.spice を生成します。その後 ngspice が batch mode で netlist を読み、raw data と CSV を生成します。

```
xschem -n -q -x - rcfile <gf180-xschemrc> \ + -o work -N nmos_dc_xschem.spice nmos_dc.sch ngspice -b work/nmos_dc_xschem.spice
```

Option	意味
-n	user config を限定して再現性を上げる
-q	quiet startup
-x	GUI を表示しない headless 実行
- rcfile	GF180 symbol/model path を設定
-o work	生成先 directory
-N file	netlist file 名
ngspice -b	対話画面なしの batch simulation

出力を確認する

1. terminal の終了 code が 0 か確認する。
2. work/nmos_dc_xschem.spice を開き、device instance と model include を探す。
3. work/nmos_dc.raw が空でないことを確認する。
4. work/nmos_id_vds.csv の header、行数、指数表記を読む。
5. 同じ directory の README.md と RESULTS.md があれば expected result と比較する。

重要 generated netlist は回路図の翻訳結果です。回路図が正しそうでも netlist の model 名、node 順、parameter が意図と違うことがあります。最初の数回は必ず両方を見ます。

8. Digital tool の使い分け

段階	Command	何を確認するか
Compile	iverilog -g2012 ...	syntax、module 接続、unsupported construct
Run	vvp work/tb	assertion/PASS、event 順序
Waveform	VCD を GTKWave 等で開く	clock、reset、state、capture、serial data

段階	Command	何を確認するか
Synthesis	yosys ... synth/abc	logic が standard cell へ写るか、cell 数
Timing	sta -exit sta.tcl	clock constraint、setup/hold slack
Physical	librelane ... config.yaml	floorplan、routing、DRC/LVS、GDS

VCD 波形の基本

1. 最初に clock と reset を表示する。
2. controller state、cell select、hit、captured code を追加する。
3. cursor を capture edge へ置き、edge 直前と直後の値を読む。
4. unknown X、高 impedance Z、意図しない glitch を探す。
5. 最後に serializer clock/data/valid を同じ timebase で確認する。

Synthesis 後に失われるもの

testbench、#delay、display、simulation 専用 real model は silicon になりません。Yosys へ渡す source list に何が含まれるかを run-digital-top.sh で確認し、mapped netlist に GF180 standard-cell instance が現れることを確認します。

Lab 1 GF180 MOS を測る

到達目標 NMOS の I-V 曲線を再生成し、回路設計に使う bias 領域を読む。

make nmos-dc

1. nmos_dc.sch を開き、device 名、W/L、body 接続、supply を確認する。
2. testbench の VGS/VDS sweep 範囲を読む。
3. make nmos-dc を実行し、CSV/plot の生成時刻を確認する。
4. 同じ VGS で VDS を上げたとき、linear 領域から saturation 領域へ移る形を説明する。

成功判定

- ☐ ngspice が error なしで終了する。
- ☐ simulations/gf180_nmos_dc/work/nmos_id_vds.csv が更新される。
- ☐ 電流の符号、単位、sweep 軸を説明できる。

設計者の問い

モデルが動くことと、回路が正しいことは別です。温度、process corner、電源範囲、device option が MPW provider の許容条件に一致しているかを必ず後段で確認します。

演習 W を 2 倍にした variant を作り、同じ bias で Id がどう変化するか予測してから simulation で確認する。

Lab 2 Sampling cell

到達目標 track/hold 動作、acquisition、droop、charge injection を波形から評価する。

make sampling-cell make four-cell make four-cell-mux

Sampling cell は入力電圧を switch で hold capacitor へ接続し、switch を切った後も 電荷として保存する回路です。

Sampling cell: voltage is converted into stored charge

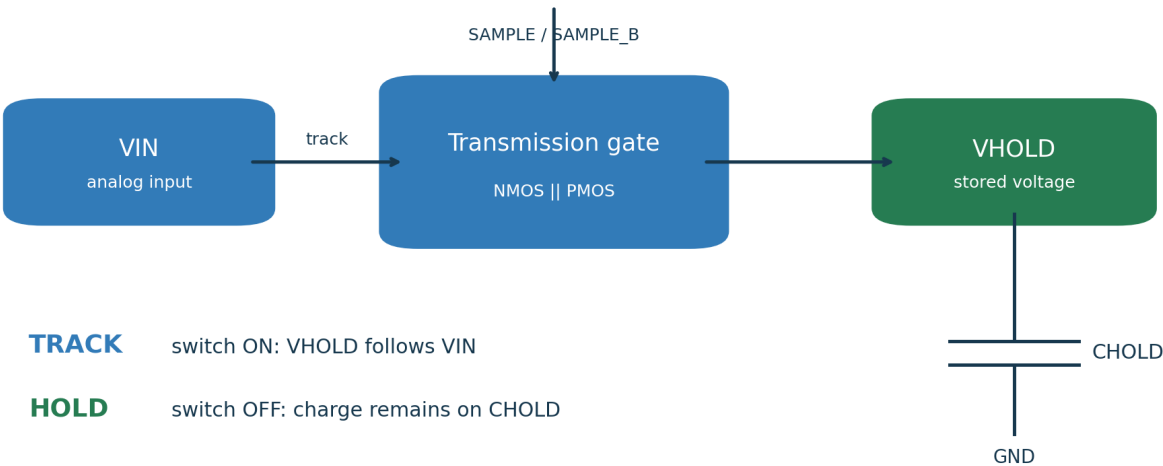


図 1 Sampling cell の構成

図 8 Transmission gate 型 sampling cell。TRACK 中は VHOLD が VIN を追跡し、HOLD 中は CHOLD に電圧を保存する。

この Lab では次の 3 回路を同じ入力条件で比較します。

回路	ファイル	確認すること
理想 switch	ideal_sampling_cell.spice	MOS 固有誤差を含まない基準
NMOS-only	sampling_cell.spice	高電圧側の on-resistance と acquisition error
Transmission gate	transmission_gate.spice	NMOS と PMOS の相補動作による改善

図 9 理想 switch、NMOS-only、transmission gate の VIN、VHOLD、SAMPLE 比較。

Xschem で transmission gate を確認する場合は、VNC の File Manager から次を開きます。

/foss/designs/simulations/gf180_sampling_cell/sampling_cell_tg.sch

NMOS gate は SAMPLE、PMOS gate は SAMPLE_B へ接続されています。Netlist、Simulate を順に押し、緑矢印を Ctrl + 左クリックして RAW を読み込みます。

sampling_cell.sch は Xschem GUI から実行できる。回路図を保存し、Netlist、Simulate の順に押した後、Load sampling waveforms を Ctrl + 左クリックする。画面には VIN、VHOLD、SAMPLE の 3 グラフが表示される。SAMPLE が high の期間は VHOLD が VIN へ追従し、low の期間は直前の電圧を保持することを確認する。

Sampling-cell simulation: ideal and GF180 switches

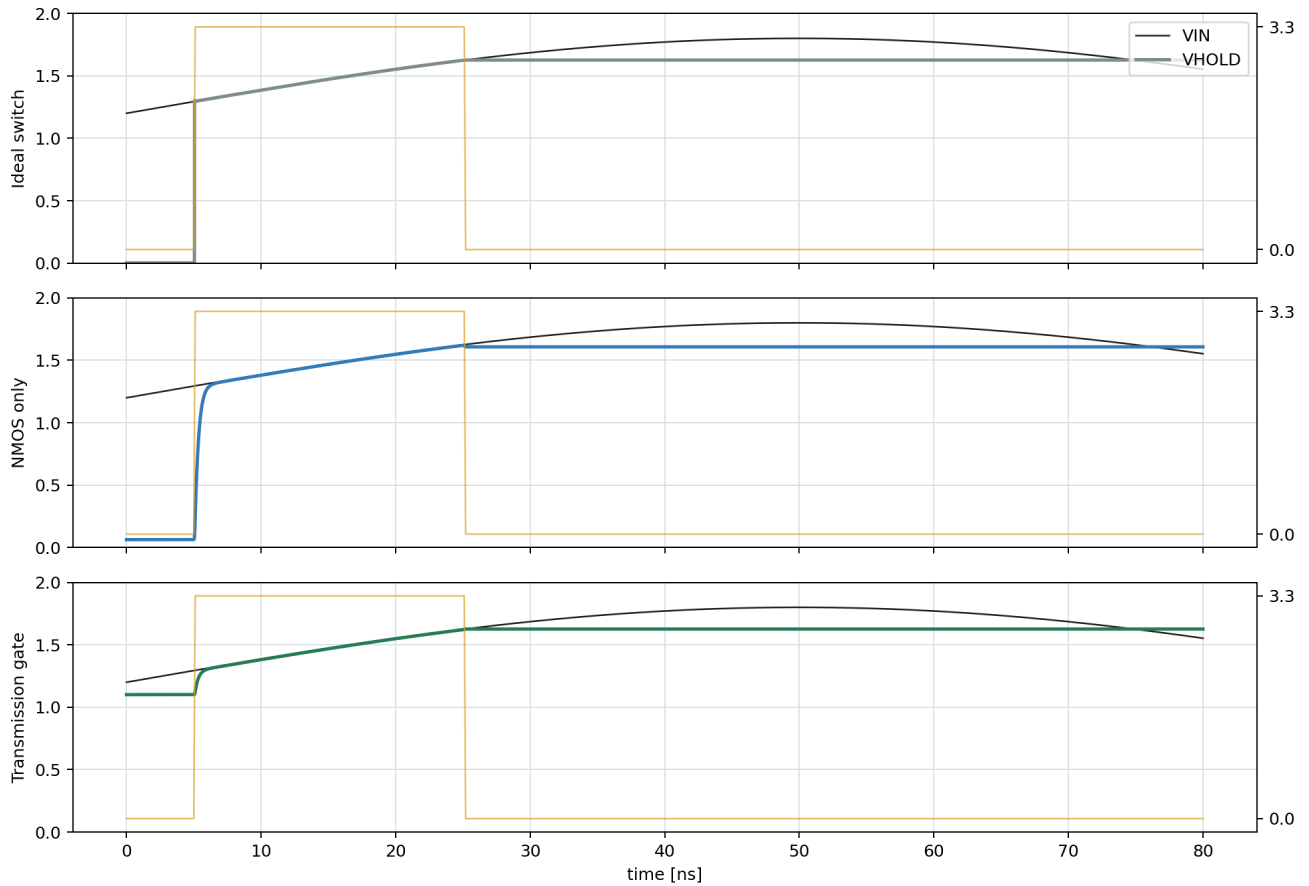


図2 3種類の sampling cell 波形

各.schに graph、launcher、GUI 用 RAW path が揃っているとは限らない。Make target は回帰試験、Xschem GUI は回路編集と波形観察という役割の違いを 意識する。

観測順序

1. SAMPLE high 中に VHOLD が VIN へ追従することを確認する。
2. SAMPLE edge 直後の step を測り、charge injection/feedthrough を区別する。
3. hold 期間の傾きを測り、droop rate [V/s] へ換算する。
4. 4-cell test で各 cell が異なる時刻の電圧を保持することを確認する。
5. MUX 選択を変え、非選択 cell の値が破壊されないことを確認する。

Cell	保持電圧 [V]	目標 [V]	誤差 [mV]
0	0.490799	0.500	9.201
1	0.792128	0.800	7.872
2	1.094365	1.100	5.635
3	1.399222	1.400	0.778

読み方 この値は prototype testbench での nominal 結果です。PVT/Monte Carlo 後の保証値ではありません。

失敗時の切り分け

- 全 cell が同じ値：sampling pulse の位相または node naming を確認。
- MUX bus が浮く：SEL decode と transmission gate の complementary control を確認。
- 保持値が急減：capacitance、leakage path、simulation timestep を確認。

Sampling cell の合格条件

- ☐ TRACK と HOLD を波形上で指し示せる。
- ☐ acquisition error、edge disturbance、hold droop の測定位置を説明できる。
- ☐ 理想 switch と MOS switch の違いを説明できる。
- ☐ transmission gate を使う理由を説明できる。

Lab 3 Comparator と ramp

到達目標入力電圧を交差時刻へ変換し、ADC の主要誤差源を説明する。

make comparator make comparator-range make comparator-offset make ramp-generator

Wilkinson 変換では、一定傾斜の ramp が保持電圧へ到達した瞬間に counter 値を capture します。理想的には $t_{\text{cross}} = (V_{\text{hold}} - V_{\text{start}}) / \text{slope}$ です。したがって ramp slope、offset、delay、noise が code へ直接写ります。

Wilkinson conversion: voltage becomes a crossing time

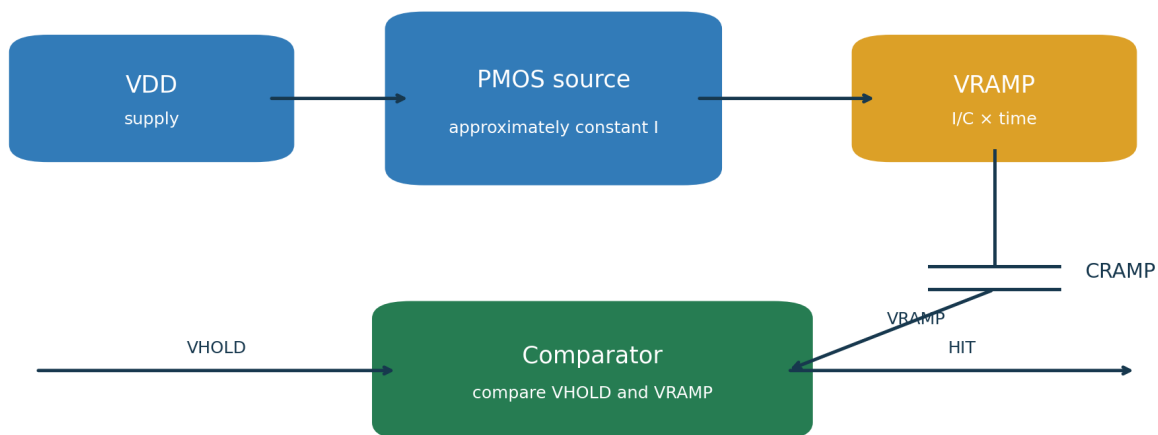


図 3 Ramp generator と comparator

図 10 一定電流で CRAMP を充電し、VRAMP と VHOLD の交差を comparator で digital edge へ変換する。

Ramp の基本式は $dV/dt = I/C$ です。電流を 2 倍にすると傾きは約 2 倍、capacitor を 2 倍にすると約半分になります。実 MOS の有限出力抵抗があるため、傾きは完全には一定になりません。

Comparator は VHOLD と VRAMP を differential pair へ入力し、active load と inverter を通して HIT を生成します。理想交差時刻と output edge の差が delay、同じ入力でも trip point がずれる静的誤差が offset です。

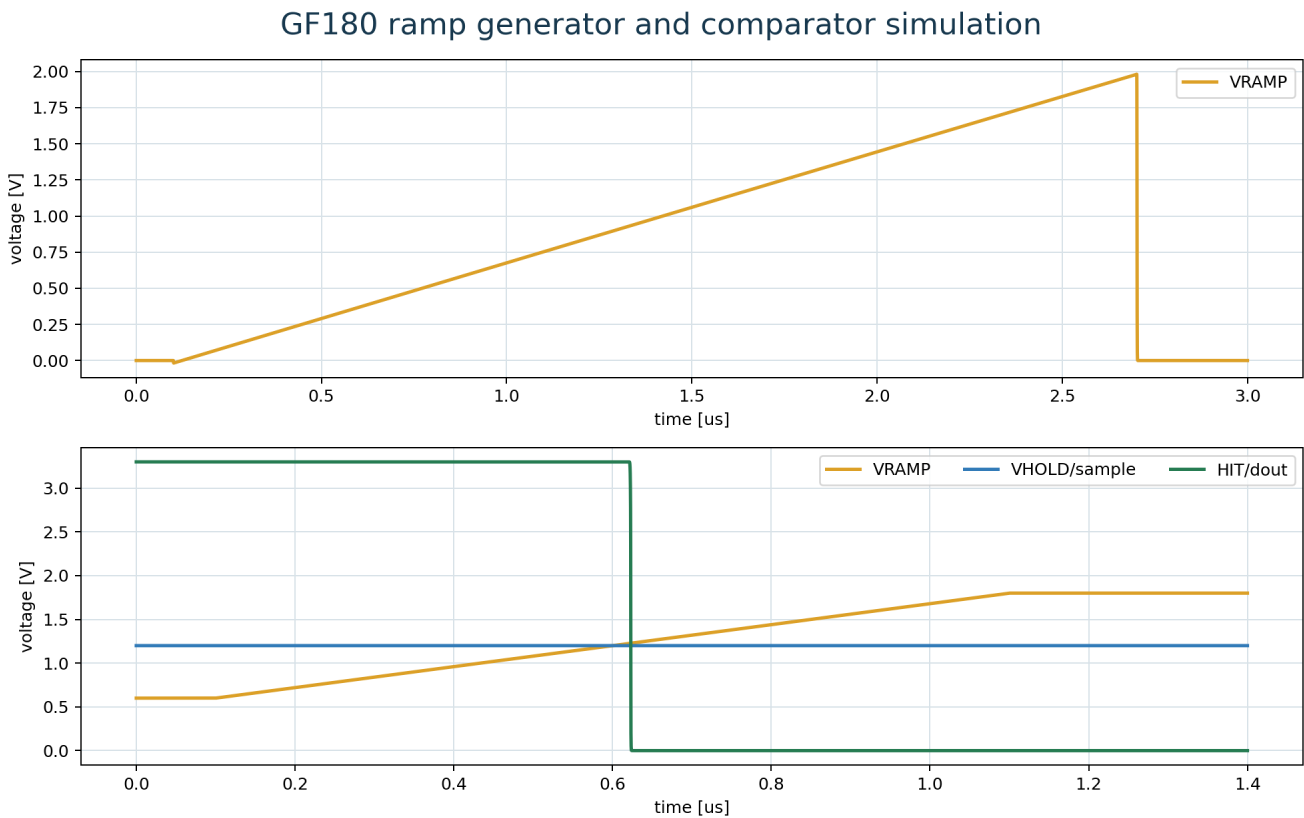


図 4 Ramp と comparator の simulation

図 11 GF180 ramp generator の reset ・ 上昇波形と、保持電圧との交差で切り替わる comparator 出力。

測るもの

Test	測定量	設計への意味
comparator	logic swing、delay	デジタル側が認識できるか
range	common-mode/input range	変換可能な analog 範囲
offset	input-referred offset	code bias、校正必要性
ramp	slope、linearity、reset	LSB 幅と conversion time

6-bit の基準

入力範囲を VFS とすると理想 LSB は VFS/64 です。1.6 V 範囲なら 25 mV/LSB です。prototype ではまず code ordering と monotonicity を確認し、INL/DNL の保証は後続版へ分離します。

演習 ramp slope を ±10% 変え、同じ入力の crossing time と code がどれだけ変わるか表にする。

Lab 4 1-cell Wilkinson slice

到達目標 sampling、ramp、comparator、counter の因果関係を 1 つの波形で説明する。

make wilkinson-slice make transfer

1. sampling phase で input を hold capacitor へ保存する。
2. conversion start で ramp reset を解除し、counter を開始する。
3. VRAMP が VHOLD へ到達すると comparator edge が発生する。
4. edge 時の counter を capture し、end-of-conversion まで保持する。
5. input sweep から code transfer を作り、単調性を確認する。

One-cell Wilkinson conversion and coarse transfer

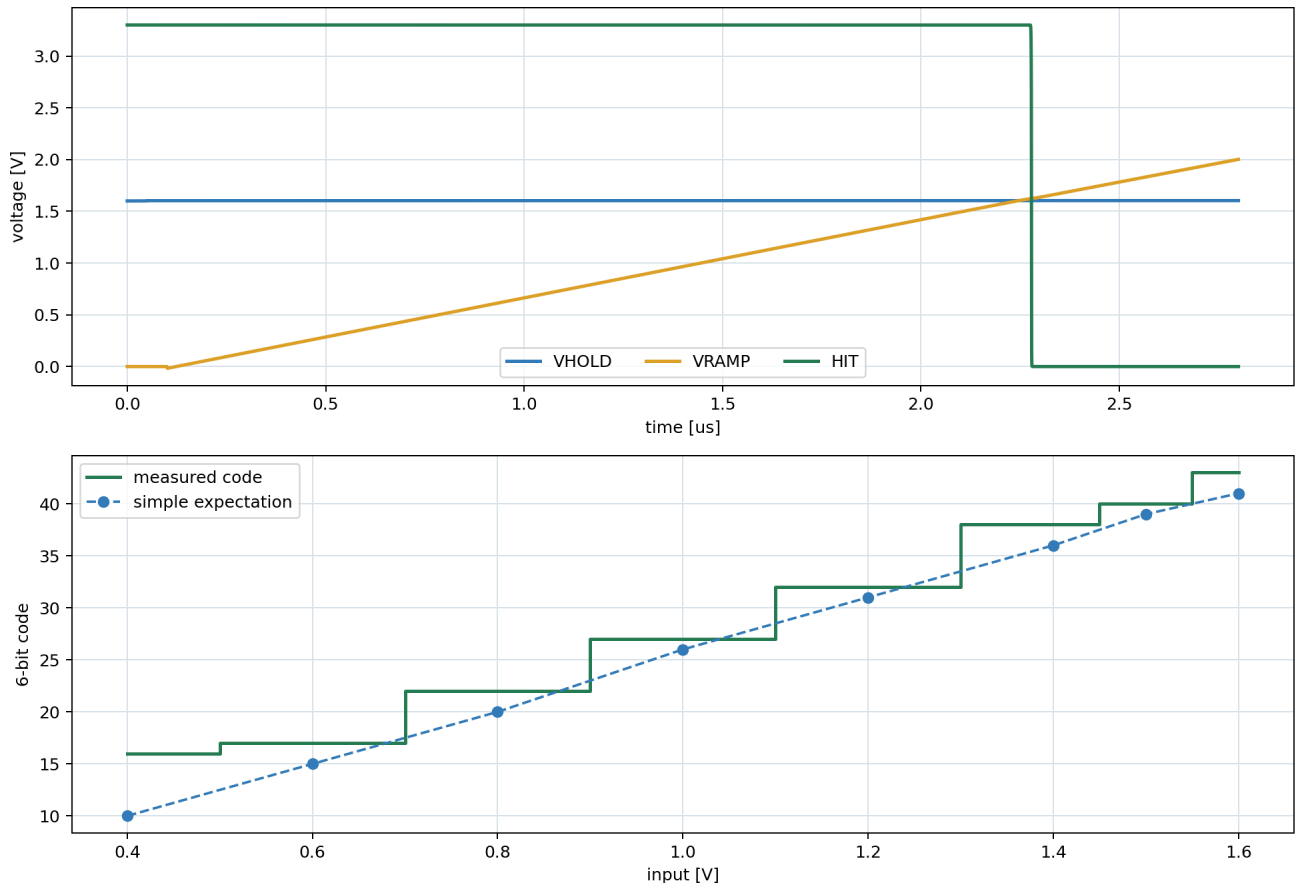


図 5 1-cell Wilkinson simulation

図 12 VHOLD、VRAMP、HIT の時間波形と、入力電圧に対する 6-bit code の粗い transfer characteristic。

合否基準

- ☐ 低い入力ほど早く交差し、小さい code になる。
- ☐ input を増やしても code が逆戻りしない。
- ☐ 0 – 63 の範囲外へ出ない。
- ☐ 飽和点と dead region を波形上で説明できる。

注意 transient 波形が自然に見えても、capture 条件が 1 timestep 依存なら脆弱です。max timestep を変えた再実行で code が不合理に変化しないことを確認します。

Lab 5 4-cell shared conversion

到達目標 4 つの保持値を 1 つの ramp/comparator で順次 6-bit code 化する。

make four-cell-wilkinson

GAW で統合波形を開く

make four-cell-wilkinson は自動合否判定に加えて、GAW で読める RAW 波形を生成します。VNC 内の Linux Terminal で次を実行します。

```
cd /foss/designs/simulations/gf180_four_cell_wilkinson
gaw work/four_cell_wilkinson.raw
```

GAW が開いたら、左側の信号名を右側の黒い graph panel へ drag and drop します。次の 3 段に分けると信号の因果関係を追いやすくなります。

- 上段: v(vin)、v(hold0)、v(hold1)、v(hold2)、v(hold3)
- 中段: v(mux_bus)、v(ramp)
- 下段: v(reset)、v(compare_out)

RAW に保存された任意の test point は GAW で表示できます。左の signal 一覧から node を選び、観測したい panel へ drag します。vin と hold0 のように因果関係を比較する signal は同じ panel へ置き、電圧 scale が異なる digital 信号は別 panel へ置きます。必要な内部 node が一覧にない場合は SPICE source の .save へ追加し、simulation を再実行します。

信号一覧が見えない場合は左側の小 window を前面に出します。波形が時間軸の一部にしか見えない場合は、上部の Z Out または Zoom menu を使います。4 cell すべてを確認するには、横軸を simulation 終了時刻の約 11.6 μ s まで表示します。

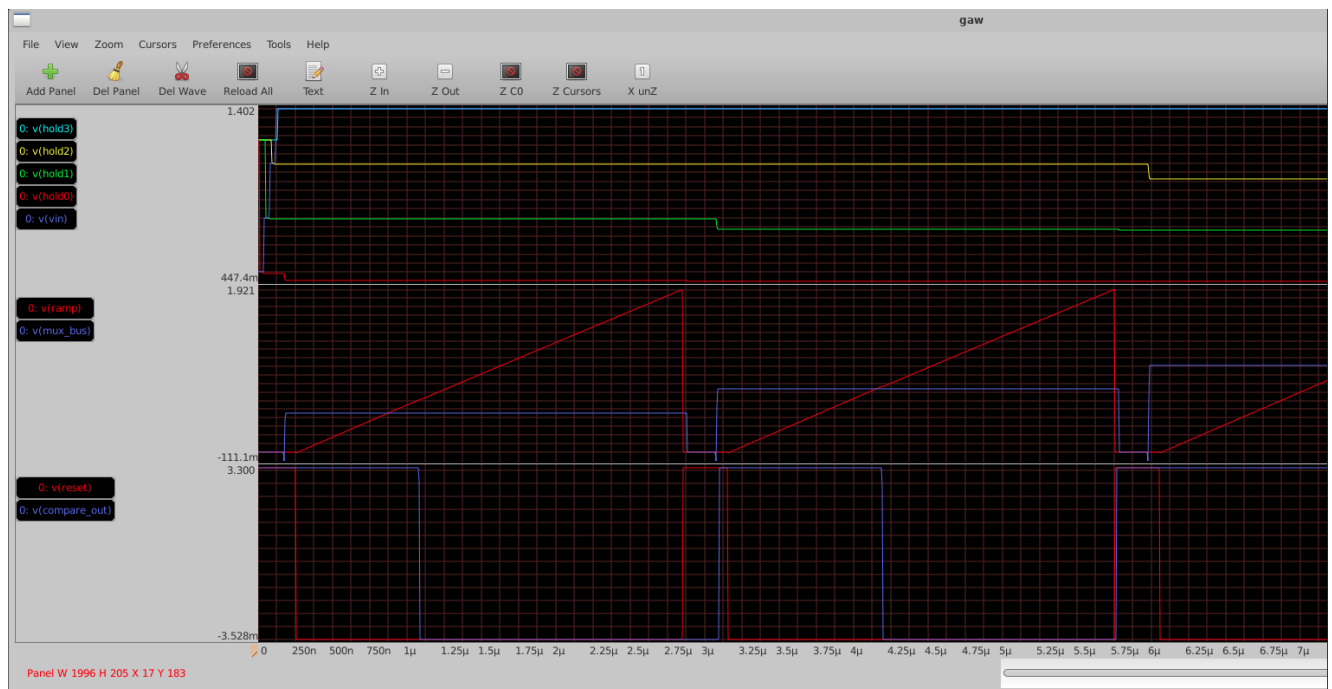


図 6 GAW で表示した 4-cell 統合 Wilkinson 波形

図 13 GAW による 4-cell 統合 SPICE 波形。上段は入力と 4 つの保持電圧、中段は MUX bus と ramp、下段は reset と comparator 出力。画像は約 7 μ s までの表示なので、最後の cell を見るときはさらに Zoom Out する。

この画面で何を検証するか

1. hold0 から hold3 がそれぞれ異なる電圧を保持し、選択されるまで大きく変化しない。
2. mux_bus が変換 slot ごとに対応する hold 電圧へ切り替わる。
3. reset の切替後、ramp が各 slot で低い電圧からほぼ直線的に上昇する。
4. ramp が mux_bus へ到達する付近で compare_out が切り替わる。
5. 高い hold 電圧ほど交差までの時間が長くなり、6-bit code も大きくなる。

この画面は波形の因果関係を人が確認するためのものです。数値の可否は Terminal に表示される `PASS: four cells complete sequential Wilkinson conversion` と、`work/measurements.txt` の `code0` から `code3` でも確認します。

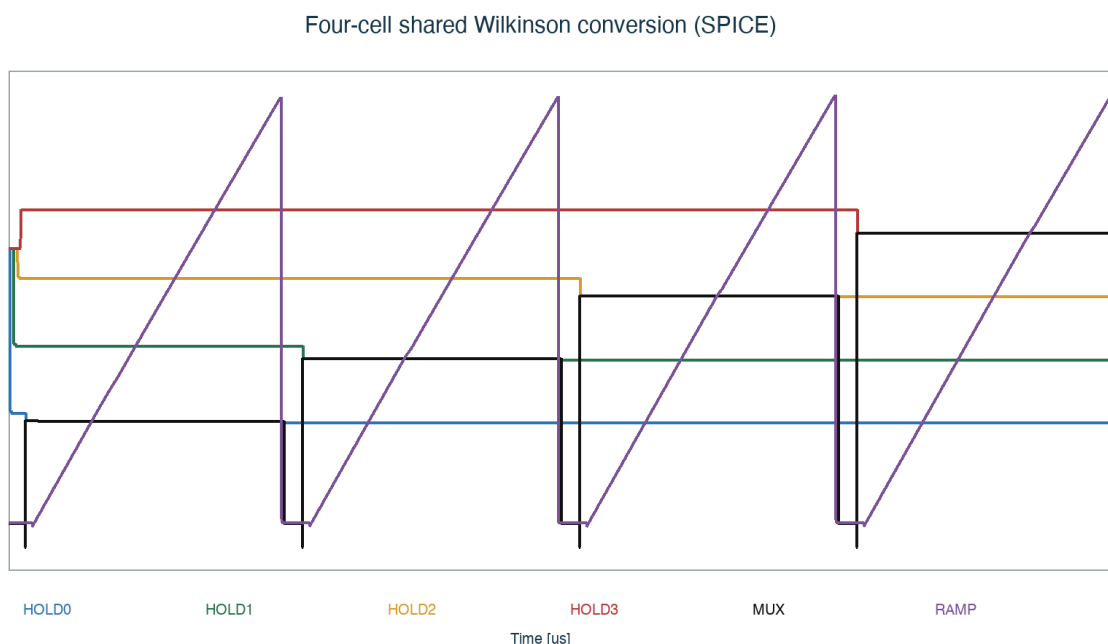


図 14 4-cell shared Wilkinson conversion。上段は保持値と MUX bus、下段は ramp と comparator。

Cell	MUX 電圧 [V]	交差時刻 [μ s]	6-bit code
0	0.456612	0.831984	16
1	0.734986	1.040260	20
2	1.014740	1.399500	27
3	1.296100	1.771850	35

code は保持値の順序に従って $16 < 20 < 27 < 35$ となっています。この段階の成功は、絶対精度よりも `sample` → `select` → `compare` → `capture` の順序とデータ対応が崩れていないことです。

レビュー観点 MUX 切替直後の settling 時間を十分取っているか。前 cell の残留値が次の crossing へ影響していないか。

Lab 6 Controller RTL

到達目標 4-cell 変換の sequencing を合成可能な RTL として検証する。

make counter make gray-counter make controller make digital-top

状態遷移を言葉で書く

1. IDLE: start を待ち、各 valid flag を clear する。
2. RESET_RAMP: ramp と counter を既知状態へ戻す。
3. SELECT: 対象 cell を MUX へ接続し settling を待つ。
4. CONVERT: Gray counter を進め、comparator hit を待つ。
5. CAPTURE: code を対象 register へ保存する。
6. NEXT/DONE: 4 cell 完了後に serializer へ引き渡す。

RTL test で見るもの

- reset 直後に X が残らない。
- cell select が one-hot または仕様どおりの binary である。
- capture は各 cell につき一度だけ発生する。
- timeout 時にも state machine が停止し続けない。
- done と data_valid の pulse/level 仕様が testbench と一致する。

設計ルール simulation 専用 delay、initial 依存、real 型を synthesizable core へ持ち込まない。analog モデルは testbench 側へ置きます。

Lab 7 SPICE-to-RTL 協調検証

到達目標 アナログ交差時刻をデジタル testbench へ渡し、end-to-end code を照合する。

make four-cell-cosim

このプロジェクトの協調検証は、SPICE が生成した crossing event を抽出し、RTL testbench へ刺激として与える方式です。アナログ simulator と RTL simulator を同時結合する重い AMS 環境を使わず、境界条件を明示できます。

境界 contract

項目	固定すべき内容
time unit	秒、ns、ps の変換規則
edge polarity	rising/falling のどちらが hit か
sampling convention	edge 直前/直後の counter を capture するか

項目	固定すべき内容
code format	binary/Gray、bit order、width
invalid case	未交差、timeout、saturation の表現

debug の順番

1. SPICE CSV に交差が存在するか。
2. event 抽出値の単位が正しいか。
3. testbench で同時刻に pulse が生成されたか。
4. capture register の enable が立ったか。
5. expected code との ± 1 境界差か、根本的な sequence 差か。

Lab 8 Gray capture と CDC

到達目標 comparator edge が counter clock 境界へ近づく現実的な不確かさを検証する。

make phase-sweep

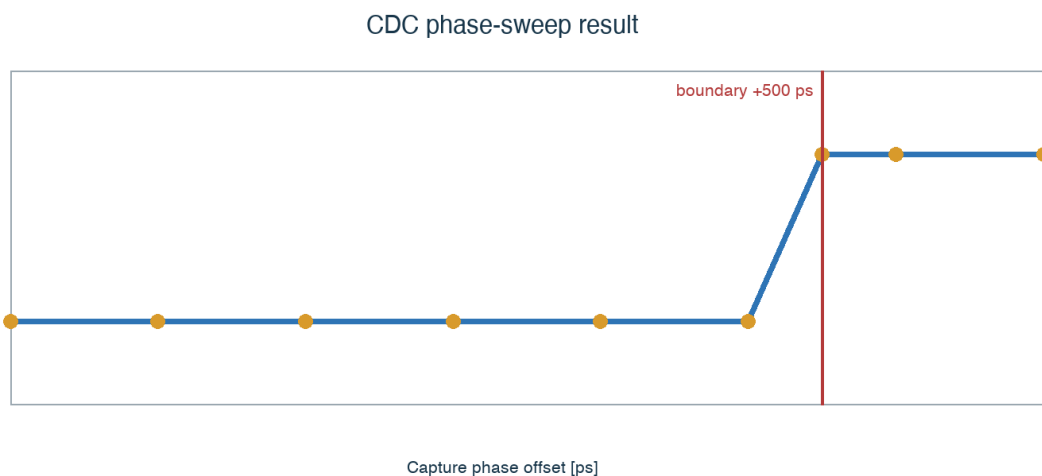


図 15 cell 2 の phase sweep. +500 ps 付近で code が 27 から 28 へ変わる。

binary counter は例として 011111 $\rightarrow$ 100000 のように複数 bit が同時に変化します。非同期 edge がその途中を capture すると存在しない code になり得ます。Gray counter は隣接 code 間で 1 bit だけ変化するため、境界付近の不確かさを原則として隣接 code へ閉じ込めます。

この結果の解釈

- +400 ps では 27、+600 ps では 28。境界で ± 1 code となることは量子化として自然。
- +500 ps は cell 2 の boundary。これは setup/hold 保証を意味しない。
- silicon では metastability、clock skew、comparator delay 分布も評価が必要。

判断記録 設計理由は docs/decisions/0003-gray-comparator-capture.md に残しています。コードだけでなく、なぜその方式を選んだかを読むこと。

9. 24-bit serial readout

4 個の 6-bit code を 24-bit word へまとめ、FPGA へ serial 転送します。pad 数を抑えられる一方、bit order、frame 境界、clock domain、idle level を仕様として固定する必要があります。

make digital-top

Field	仕様確認
Payload	code0, code1, code2, code3 の連結順
Bit order	MSB-first または LSB-first
Handshake	valid/ready、busy、done
Clock	ASIC 生成か FPGA 供給か
Reset	途中 reset 時の line 状態

logic analyzer での受入試験

1. 既知 code 16, 20, 27, 35 を load する。
2. serial clock と data を同時取得する。
3. 24 edge 分を decode し、元の 4 code へ戻す。
4. frame を連続送信し、境界で bit slip がないことを確認する。

PCB へつながる仕様 I/O voltage、drive strength、level shifting、ground reference、connector pinout は pad library と評価 PCB の制約に依存します。prototype RTL の論理仕様だけでは確定しません。

Lab 9 RTL-to-GDS

到達目標合成、floorplan、placement、CTS、routing、signoff の成果物を確認する。

make digital-physical

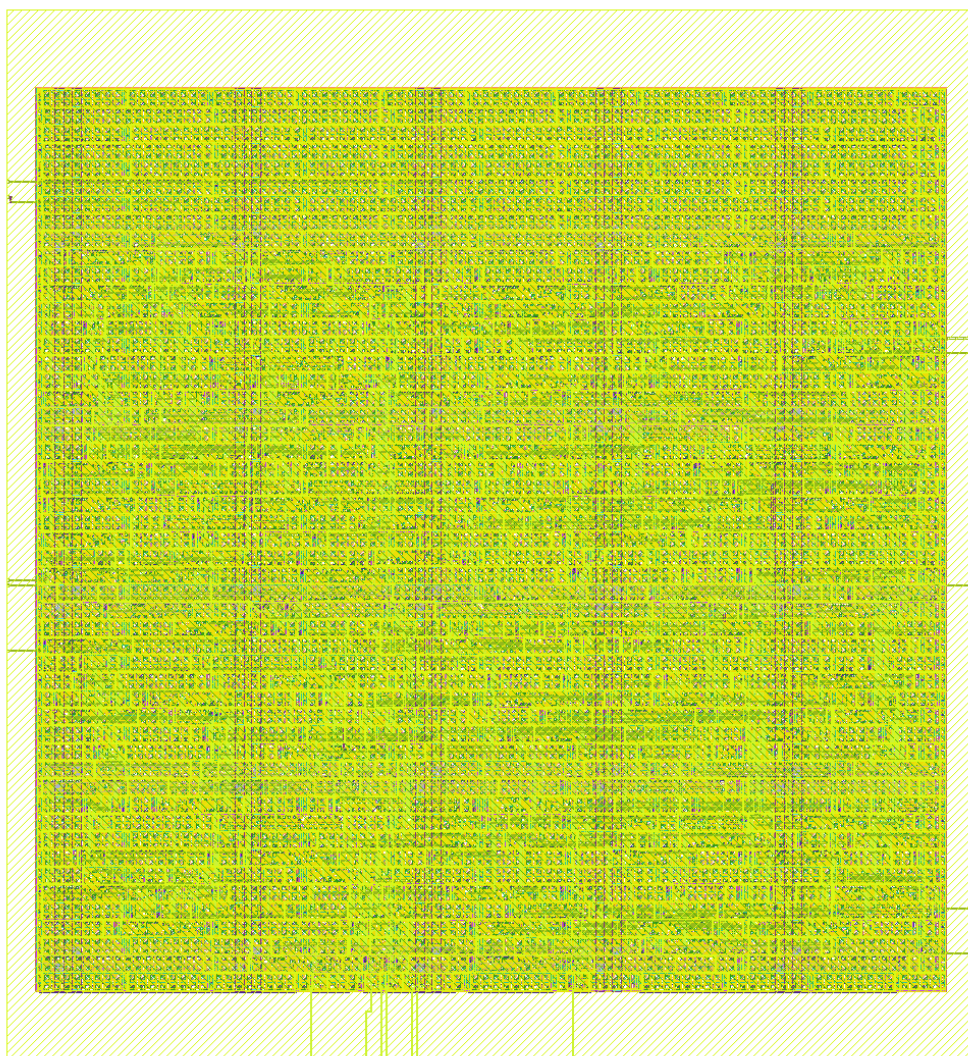


図 16 GF180 standard-cell flow で生成したデジタル top の最終レイアウト。

Metric	結果
Core/die size	215.49 × 233.41 μm
Die area	50,297.5 μm^2
Utilization	39.5%
Standard cells	798 (sequential 101)
Total wire / vias	14,792 μm / 2,784
Estimated power	6.13 mW

成果物は digital/asic_digital_top/physical/final_views/ 以下にあります。GDS、DEF、LEF、gate-level netlist、metrics を相互に対応づけます。

9.1 Analog layout は別に設計する

Digital block は standard cell を自動配置配線できますが、sampling cell、MUX、ramp、comparator は device matching と寄生を考えて手作業で layout します。

Sampling cell:

- NMOS/PMOS の clock 配線を短くする。
- VHOLD node を短くし、digital 配線から離す。
- CHOLD 周辺の coupling を抑える。
- guard ring と well connection を確認する。

Comparator:

- differential pair を対称配置する。
- input 配線長を揃える。
- current mirror を matching 配置する。
- digital output buffer を input pair から離す。

Ramp:

- ramp node の寄生容量を管理する。
- digital clock から shield する。
- current-source device の周囲環境を揃える。

9.2 DRC、LVS、PEX の順序

schematic simulation

→ layout

→ DRC

→ extraction

→ LVS

→ post-layout simulation

- DRC: layout が foundry の geometry rule を守るか確認する。
- LVS: layout から抽出した接続が schematic と一致するか確認する。
- PEX: 配線抵抗、寄生容量、coupling を含む netlist を生成する。

LVS PASS だけでは性能は保証されません。PEX 後に sampling error、ramp slope、comparator delay を同じ測定定義で再確認します。

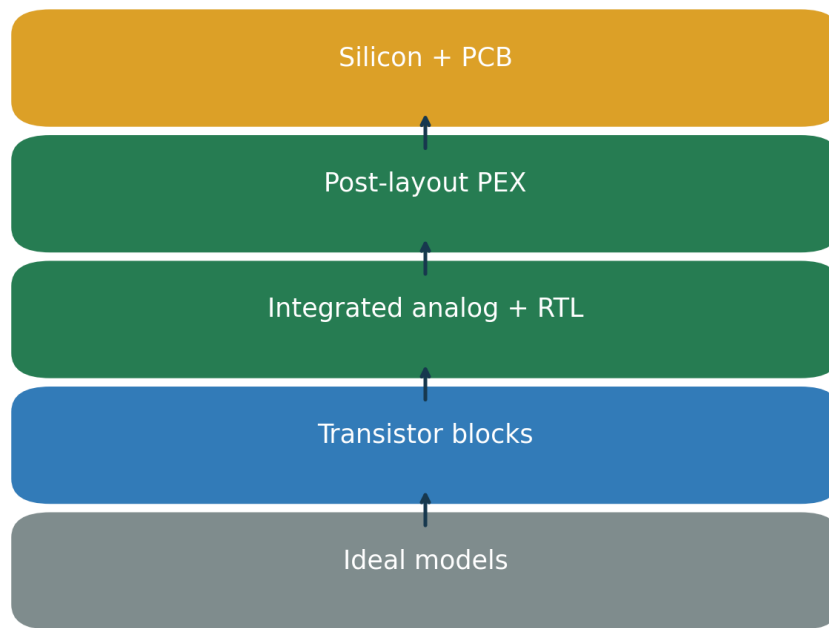
図 17 検証は理想 model、transistor block、統合回路、PEX、silicon の順に積み上げる。

10. Signoff 結果の読み方

Check	今回の結果	意味
DRC	0 violations	実行 deck 上の layout rule に違反なし
Antenna	0 violations	報告対象の antenna rule 違反なし
LVS	0 violations	layout と参照 netlist の接続一致
Setup	0 violations	解析 corner で setup requirement を満足
Hold	0 violations	解析 corner で hold requirement を満足
Worst setup slack	39.21 ns	現在の clock constraint に大きな余裕
Worst hold slack	0.336 ns	最小 hold 余裕

重要 0 violations は『チップ全体が tape-out ready』を意味しません。今回の結果はデジタルブロック、使用した library/view、設定した constraint の範囲で成立します。

Verification is a ladder, not one giant simulation



Every level answers a different question

図7 段階的な検証

証拠を確認する

- metrics.json/csv: 数値結果。
- asic_digital_top.gds: foundry へ渡す形状データ候補。
- asic_digital_top.nl.v: 物理実装後の gate-level netlist。
- resolved.json: 実際に解決された flow 設定。
- run log/report: warning と waiver を含む実行証跡。

確認問題

- clock period を半分にした場合、setup slack がどう変わるか予測する。
- hold slack が正でも安心しきれない理由を corner、OCV、I/O constraint から説明する。

11. 再現可能なデバッグ

エラーを見たらランダムに設定を変えず、最初に失敗した境界を特定します。下流のエラーは上流の欠損から連鎖していることが多いからです。

標準 debug loop

1. 再現：同じ `commit` と `command` で再発させる。
2. 縮小：最小 `testbench` または単一 `cell` へ戻す。
3. 観測：input、internal state、output を同じ `timebase` で保存する。
4. 仮説：1 回の実行で検証できる原因を 1 つ書く。
5. 変更：1 要因だけ変える。
6. 回帰：直った `test` と既存 `test` を両方実行する。
7. 記録：原因、変更、証拠を `commit` または `decision log` へ残す。

症状	最初に見る場所
SPICE convergence code が全部 0/63	initial condition、ramp discontinuity、timestep、floating node
cell 順が入れ替わる	comparator polarity、reset、range
±1 が不安定	MUX select、payload packing
RTL sim は PASS、P&R 失敗	capture convention、CDC boundary
DRC/LVS 不一致	clock/reset、unsupported construct、constraints
	PDK deck、pin label、bulk/substrate、netlist source

12. 実践演習

演習 A：再現レポート

- `make check` から `make four-cell-wilkinson` まで実行する。
- `commit hash`、実行環境、`command`、主要波形、判定を 2 ページでまとめる。
- nominal 結果と保証値を混同していないか自己レビューする。

演習 B：ADC 設計変更

- ramp slope を変更し、conversion time と code scale の変化を予測する。
- simulation を実行し、予測との差を説明する。
- VERIFICATION_MATRIX へ新しい `test` を 1 行追加する。

演習 C：CDC boundary

- phase sweep 点を増やし、遷移境界を絞り込む。
- binary capture 版と Gray capture 版の failure mode を比較する。
- ±1 code を許容する判定条件を文章で定義する。

演習 D : physical experiment

- clock constraint または utilization を 1 つだけ変える。
- area、slack、wire length、runtime の変化を比較する。
- 改善した metric と悪化した metric を両方報告する。

提出物 README 形式の短い実験記録、変更 diff、再現 command、主要 artifact への相対 path。画像だけの提出は不可。

13. Prototype から tape-out へ

このリポジトリは complete design flow の demonstration として大きな節目に到達しています。しかし foundry へ提出する full chip には、デジタル GDS 以外の設計と provider 固有条件が残っています。

残作業	必要な出口条件
MPW/provider 確定	run 日程、PDK release、submission checklist、面積、公開条件
Analog layout	sampling/MUX/ramp/comparator の layout、PEX、DRC/LVS
PVT/Monte Carlo	corner、温度、電源、mismatch の pass/fail
Top integration	analog + digital floorplan、power/ground、clock、substrate strategy
Pad/ESD	provider-qualified I/O library、voltage、ESD、latch-up
Package/PCB	bond map、package parasitic、evaluation PCB、FPGA interface
Full-chip signoff	top DRC/LVS、antenna、ERC、density、timing、deliverable audit

電源上の注意 現デジタル flow は GF180 の gf180mcu_fd_sc_mcu7t5v0 timing views を使用しています。最終電源、pad、library 選択は MPW provider の qualified option 確認後に freeze します。

Tape-out readiness review

- ☐ PDK と tool version が固定されている。
- ☐ 全要求に test と artifact が紐づいている。
- ☐ 全 waiver に owner と根拠がある。
- ☐ pad ring、seal ring、density/fill を含む top GDS がある。
- ☐ package/PCB/test plan が chip pinout と一致する。

高性能化すると何が難しくなるか

Sampling rate を上げると、sampling aperture、switch settling、clock skew、input bandwidth、bus capacitance、clock feedthrough が厳しくなります。Resolution を上げると、comparator offset/noise、ramp linearity、counter clock jitter、pedestal variation、supply noise、calibration 精度が厳しくなります。

1 V 入力範囲での理想 LSB は次の通りです。

Resolution	Levels	Ideal LSB
6 bit	64	15.625 mV
8 bit	256	3.906 mV
10 bit	1024	0.977 mV
12 bit	4096	0.244 mV

12-bit、1 GSa/s は open-source tool を使うだけで自動的に達成できる仕様ではありません。回路、layout、clock、package、PCB、calibration を含む system として段階的に実証します。

14. Specification snapshot

項目	Prototype v0
Process	GF180MCU Open PDK baseline
Channels	1 analog channel
Storage depth	4 cells
ADC	shared Wilkinson, 6 bit
Readout	24-bit serial payload
Control	synthesizable RTL FSM
Primary objective	complete reproducible design flow
Not yet claimed	target bandwidth, precision, radiation tolerance, production yield

用語集

用語	意味
PDK	process の device model、layer、rule deck、library 等
MPW	複数 design で wafer cost を共有する shuttle
SCA	switched-capacitor array。波形を capacitor 列へ保存
Wilkinson ADC	ramp の交差時刻を counter で測る ADC
CDC	clock domain crossing。非同期信号の受け渡し
DRC	layout 形状が design rule へ適合するか
LVS	layout 接続と schematic/netlist の一致
STA	ベクトルを使わず path timing を検証
PEX	layout 寄生 R/C を抽出して再 simulation
GDSII	mask layout を表す交換形式

15. Command quick reference

Command	対象
make check	environment/repository sanity
make nmos-dc	GF180 NMOS DC sweep

Command	対象
make sampling-cell	single sampling cell
make four-cell	4-cell sampling array
make comparator	comparator transient
make comparator-range	input range
make comparator-offset	offset sensitivity
make ramp-generator	ramp
make wilkinson-slice	single 6-bit conversion
make transfer	ADC transfer
make four-cell-mux	shared analog MUX
make four-cell-wilkinson	4-cell integrated analog
make gray-counter	Gray counter RTL
make controller	conversion controller
make four-cell-cosim	SPICE-to-RTL co-sim
make phase-sweep	CDC boundary sweep
make digital-top	full RTL regression
make digital-physical	RTL-to-GDS
make vnc	browser desktop

16. Final review sheet

以下を口頭または README で説明できれば、この prototype の complete flow を自走できています。

- ☐ なぜ 4-cell/6-bit まで仕様を緩めたのか。
- ☐ sampling error が ADC code へどう伝播するか。
- ☐ ramp/comparator/counter がどの順番で働くか。
- ☐ なぜ Gray capture を選んだのか。
- ☐ SPICE と RTL の境界 contract は何か。
- ☐ DRC/LVS/STA の 0 violations が何を保証し、何を保証しないか。
- ☐ どのファイルが source で、どれが generated artifact か。
- ☐ tape-out までに残る top-level blocker は何か。

次の設計課題 最優先は analog layout + extracted simulation です。sampling cell を最初の layout 対象とし、schematic → layout → DRC → LVS → PEX → transient 比較を 1 ブロックで完結させます。

主要参照先

- GitHub: https://github.com/sykeisuke/asic_rd
- docs/PROTOTYPE_SPECIFICATION.md
- docs/VERIFICATION_MATRIX.md

- docs/TAPEOUT_BLOCKERS.md
- docs/decisions/0003-gray-comparator-capture.md

このハンドブック自体も設計成果物です。回路、**command**、結果、判断が変わったら、同じ **commit** で更新してください。